D1 - Foundation Modules:

1. Planning Logic – Pseudocode & Flowcharts

🔹 What is Planning Logic?

Before writing actual code, we **plan the logic** — the sequence of steps a program should perform to achieve a result.

This avoids errors and helps in clear thinking.

🔹 Pseudocode

Pseudocode is **plain English-like code** that represents the logic before actual programming.

It's not syntax-bound — the goal is clarity.

Example: Find the largest of two numbers

```
START
READ A, B
IF A > B THEN
    PRINT "A is larger"
ELSE
    PRINT "B is larger"
END IF
STOP
```

🔹 Flowchart

A **flowchart** is a **visual representation** of the logic.

```
        ┌──────────────┐
        │   Start      │
        └──────┬───────┘
               │
          ┌────▼─────┐
          │ Input A,B │
          └────┬─────┘
               │
          ┌────▼─────┐
          │ A > B ?   │
          └────┬─────┘
          Yes / \ No
```

```
           /   \
     ┌───────▼     ▼───────┐
     │Print A>Larger│Print B>Larger│
     └──────────────┘
             │
         ┌───▼────┐
         │   End   │
         └────────┘
```

Industry Use:

Used in **software design**, **system architecture**, and **algorithm development** to communicate logic across teams before implementation.

---

2. Programming Basics – What is Programming? Types of Languages

🔹 What is Programming?

Programming is **instructing a computer** to perform specific tasks by writing **logical, structured commands** in a programming language.

🔹 Types of Programming Languages

Type	Description	Example
Low-Level	Closer to hardware; difficult but fast	Assembly
High-Level	Human-readable; portable	Python, Java, C++
Procedural	Step-by-step instructions	C
Object-Oriented (OOP)	Uses objects & classes	Java, Python
Scripting	Automates small tasks	Python, JavaScript
Functional	Based on functions & recursion	Haskell, Scala

Industry Use:

Python is preferred for **data, AI, automation**, Java for **enterprise apps**, and JavaScript for **web development**.

---

3. Syntax & Structure – Variables, Data Types, Operators

🔹 Syntax

Syntax = **rules** of the programming language.

Example (Python):

```
print("Hello, World!")    # Correct
Print("Hello, World!")    # Error (case-sensitive)
```

🔹 Variables

A **variable** stores a value in memory.

```
name = "Dinesh"
age = 25
```

🔹 Data Types

Type	Example	Description
int	10	Integer number
float	10.5	Decimal number
str	"Hello"	Text
bool	True/False	Logical
list	[1,2,3]	Collection

🔹 Operators

Type	Example	Description
Arithmetic	+, -, *, /	Math ops
Comparison	>, <, ==, !=	Compare values
Logical	and, or, not	Combine conditions
Assignment	=, +=, -=	Assign values

Example:

```
a, b = 10, 5
print(a + b)    # 15
print(a > b)    # True
```

---

4. Control Structures – if, else, loops (for, while)

These control how code executes — **decision-making & repetition.**

🔹 if–else (Decision)

```python
marks = 75
if marks >= 60:
    print("Pass")
else:
    print("Fail")
```

Pseudocode:

```
IF marks >= 60 THEN
    PRINT "Pass"
ELSE
    PRINT "Fail"
```

Flowchart:

```
   ┌─────────────┐
   │  Start      │
   └──────┬──────┘
          │
     ┌────▼─────┐
     │ Input marks │
     └────┬─────┘
          │
     ┌────▼─────┐
     │ marks>=60? │
     └────┬─────┘
      Yes / \ No
         /   \
 ┌──────▼─┐ ┌▼──────┐
 │ Print Pass │ Print Fail │
 └────────┘
          │
     ┌────▼─────┐
     │  End     │
     └──────────┘
```

🔹 Loops (Repetition)

For Loop (fixed repetition)

```
for i in range(1, 6):
    print("Iteration:", i)
```

→ Prints 1 to 5

While Loop (condition-based)

```
count = 1
while count <= 5:
    print("Count:", count)
    count += 1
```

Industry Use:

Used for **data iteration, user input validation, repetitive tasks**, etc.

---

5. Functions & Modular Programming

🔹 Why Functions?

Functions **modularize** code — make it reusable, organized, and easy to test.

Definition:

```
def greet(name):
    return "Hello, " + name
```

Calling:

```
message = greet("Karthik")
print(message)
```

Output:

```
Hello, Karthik
```

🔹 Parameters and Return Values

- **Parameters**: inputs to the function
- **Return value**: output from the function

```python
def add(a, b):
    return a + b

result = add(10, 20)
print(result)  # 30
```

Pseudocode:

```
FUNCTION add(a, b)
    RETURN a + b
END FUNCTION
```

🔹 Modular Programming

Breaking the program into **smaller, independent modules** (functions or files).

Example Project: Calculate Student Grades

```python
def input_marks():
    return [85, 78, 92]

def calculate_average(marks):
    return sum(marks) / len(marks)

def display_result(avg):
    if avg >= 60:
        print("Pass")
    else:
        print("Fail")

# Main module
marks = input_marks()
average = calculate_average(marks)
display_result(average)
```

✅ Reusable

✅ Easy to debug

✅ Industry-standard structure

Summary Table

Concept	Purpose	Example
Pseudocode	Plan logic	Steps to find largest number
Flowchart	Visual logic	Diamonds = decision
Programming Basics	Instruct computer	Python, Java, etc.
Variables & Data Types	Store data	name = "John", age = 25
Operators	Perform actions	+, -, ==
Control Structures	Decision & loops	if, for, while
Functions	Reusable blocks	def add(a, b)